Guía paso a paso
Tipos de datos en TypeScript con Angular

Registro paso a paso de la construcción del proyecto de ejemplo: interfaces y tipado en TypeScript, y componentes standalone, data binding, control de flujo (@if, @for), signals y reactividad sin zone.js en Angular.

Introducción

Este documento recoge, en orden, todos los pasos seguidos para construir este proyecto de ejemplo. El objetivo es introducir a los alumnos en TypeScript (interfaces, enumerados, tipado) y en Angular (componentes standalone, data binding, control de flujo con `@if` y `@for`, signals y reactividad sin zone.js). Cada parte corresponde a un commit con su etiqueta: al hacer checkout de una etiqueta, el código y esta guía muestran el estado de ese paso.

La primera parte (commit `01_tipado`, pasos 1 a 6) define los tipos `Product`, `Rating` y `Category`, y muestra un único producto del array `PRODUCTS`, **sin usar todavía directivas de control de flujo** (`@if`, `@for`).

La segunda parte (commit `02_busqueda`, pasos 7 a 14) amplía el componente `Products` con una búsqueda de productos por id, e introduce `@if`, las variables de plantilla y el two-way data binding, sin usar formularios ni signals.

La tercera parte (commit `03_reactividad`, pasos 15 a 25) explica el **estado y la reactividad en Angular sin zone.js (zoneless)** e introduce los signals y `@for`. Primero, con el componente `Reactividad`, se compara un número guardado en una propiedad normal y otro guardado en un signal. Después, con dos componentes casi iguales, `ProductsArray` y `ProductsSignal`, se hace la misma comparación con un array de productos.

Pasos seguidos

1. Estructura de carpetas `core` / `shared` / `features`

Dentro de `src/app` se creó la organización habitual en proyectos Angular:

```
src/app/
  core/
    services/        # servicios singleton (providedIn: 'root')
    guards/          # guards de rutas
    interceptors/    # interceptores HTTP
    models/          # interfaces y tipos compartidos
    mocks/           # datos de ejemplo estaticos
  shared/
    components/      # componentes reutilizables
    directives/      # directivas reutilizables
    pipes/           # pipes reutilizables
  features/
    products/        # funcionalidad "productos"
```

Cada carpeta vacía se creó con un `.gitkeep` para que Git la trackee.

2. Enumerado `Category` e interfaces `Product` y `Rating`

Antes de definir la interfaz `Product` se revisaron los datos de fakestoreapi.com/products y se observó que el campo `category` solo toma 4 valores distintos en los 20 productos: "men's clothing", "women's clothing", "jewelery" y "electronics". En vez de tipar ese campo como `string` libre, se creó primero un enum que restringe `category` a esos valores concretos:

- **`core/models/category.model.ts`**

```
export enum Category {
  MenClothing = "men's clothing",
  WomenClothing = "women's clothing",
  Jewelery = 'jewelery',
  Electronics = 'electronics'
}
```

Se usa un **enum de tipo string** (cada miembro tiene asignado el valor de texto real de la API) en vez de un enum numérico, para que el valor mostrado en pantalla siga siendo el texto original (`"electronics"`, etc.) sin necesidad de convertirlo. Si más adelante alguien escribe mal una categoría (por ejemplo `Category.Eletronics`), el compilador de TypeScript lo marca como error en vez de dejarlo pasar como un `string` cualquiera.

Con `Category` ya definido, se crearon las dos interfaces TypeScript en `core/models`:

- **`core/models/rating.model.ts`**

```
export interface Rating {
  rate: number;
  count: number;
}
```

- **`core/models/product.model.ts`**

```
import { Category } from './category.model';
import { Rating } from './rating.model';

export interface Product {
  id: number;
  title: string;
  price: number;
  description: string;
  category: Category;
  image: string;
  rating: Rating;
}
```

`Product` compone `Rating` y `Category` como propiedades anidadas, para mostrar cómo se combinan interfaces y enumerados en TypeScript.

3. Array de datos de ejemplo (`PRODUCTS`)

En lugar de hacer una petición HTTP a la API, se pidió tener los datos como una **constante local**. Se consultó el contenido real de `https://fakestoreapi.com/products` y se copió en:

- **`core/mocks/products.mock.ts`**

```
import { Category } from '../models/category.model';
import { Product } from '../models/product.model';
```

```
export const PRODUCTS: Product[] = [
  {
    id: 1,
    title: '...',
    price: 109.95,
    /* ... */
    category: Category.MenClothing,
    rating: { rate: 3.9, count: 120 }
  },
  // ...20 productos en total
];
```

Este array está tipado explícitamente como `Product[]`, de forma que el compilador valida que cada objeto cumple la interfaz.

No se configuró `HttpClient` ni se tocó `app.config.ts`: el ejemplo es 100% estático, pensado solo para trabajar con tipos.

4. Imágenes en local

Para no depender de la red al mostrar el ejemplo, se descargaron las 20 imágenes de producto desde `fakestoreapi.com` y se guardaron en:

```
public/images/{id}.png
```

(por ejemplo, `public/images/11.png` para el producto con `id: 11`).

Después se actualizó el campo `image` de cada producto en `products.mock.ts` para que apunte a la ruta local:

```
image: '/images/11.png',
```

5. Componente `Products`

Se creó un componente standalone en `features/products`:

- `products.ts`
- `products.html`
- `products.css`

```
import { ChangeDetectionStrategy, Component } from '@angular/core';
import { NgOptimizedImage } from '@angular/common';

import { PRODUCTS } from '../../core/mocks/products.mock';
import { Product } from '../../core/models/product.model';

@Component({
  selector: 'app-products',
  imports: [NgOptimizedImage],
  templateUrl: './products.html',
  styleUrl: './products.css',
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class Products {
  protected readonly product: Product = PRODUCTS[10];
}
```

Puntos clave para los alumnos:

- El componente selecciona **un único producto** del array (`PRODUCTS[10]`) accediendo por índice, sin necesidad de directivas como `@for`.
- `product` está tipado como `Product`, así que el editor autocompleta y valida sus propiedades (`title`, `price`, `rating.rate`, etc.).
- Se usa `ChangeDetectionStrategy.OnPush`, como marca la guía de estilo del proyecto.
- La imagen usa `NgOptimizedImage` (`[ngSrc]`) en vez de `<img src>`, ya que ahora es un asset estático servido desde `public/`.

La plantilla (`products.html`) simplemente interpola las propiedades del objeto `product`:

```
<section aria-label="Producto">
  <h1 class="products-title">Producto {{ product.id }}</h1>
  <article class="product-card">
    <img [ngSrc]="product.image" [alt]="product.title" width="220" height="220" class="product-
card__image" />
    <h2 class="product-card__title">{{ product.title }}</h2>
    <p class="product-card__category">{{ product.category }}</p>
    <p class="product-card__price">{{ product.price }} €</p>
    <p class="product-card__rating">
      ★ {{ product.rating.rate }} ({{ product.rating.count }} valoraciones)
    </p>
  </article>
</section>
```

6. Montaje en `AppComponent`

Se sustituyó la plantilla de bienvenida que genera `ng new` por la llamada al componente `Products`, manteniendo el `<router-outlet />` para futuras rutas:

- **`app.ts`**

```
import { ChangeDetectionStrategy, Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

import { Products } from './features/products/products';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Products],
  templateUrl: './app.html',
  styleUrl: './app.css',
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class App {}
```

- **`app.html`**

```
<app-products />
<router-outlet />
```

Segunda parte: búsqueda de productos por id (02_busqueda)

En la segunda parte (commit `02_busqueda`) el componente `Products` deja de mostrar un producto fijo (`PRODUCTS[10]`) y pasa a **buscar un producto por su id**. La búsqueda se hace de tres formas distintas, colocadas una al lado de la otra, para comparar cómo llega un valor de la vista al componente.

Esta parte se hace **sin formularios** (`<form>`, Reactive Forms) **y sin signals**: el estado se guarda en propiedades normales de la clase. La única excepción es `FormsModule`, que se importa solo para poder usar `[(ngModel)]`. Es también la primera vez que se usa una directiva de control de flujo (`@if`).

7. El componente de búsqueda (`products.ts`)

La propiedad `product` de la primera parte se sustituye por el estado de la búsqueda y por los métodos que usa cada forma:

- **`features/products/products.ts`**

```
import { ChangeDetectionStrategy, Component } from '@angular/core';
import { NgOptimizedImage } from '@angular/common';
import { FormsModule } from '@angular/forms';

import { PRODUCTS } from '../../core/mocks/products.mock';
import { Product } from '../../core/models/product.model';

@Component({
  selector: 'app-products',
  imports: [NgOptimizedImage, FormsModule],
  templateUrl: './products.html',
  styleUrl: './products.css',
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class Products {
  protected selectedArticulo: Product | undefined = undefined;
  protected searched = false;
  protected searchId: number | null = null;
  protected searchIdManual: number | null = null;

  protected search(id: string): void {
    this.findArticulo(Number(id));
  }

  protected searchById(): void {
    this.findArticulo(this.searchId);
  }

  protected updateSearchIdManual(event: Event): void {
    const value = (event.target as HTMLInputElement).value;
    this.searchIdManual = value === '' ? null : Number(value);
  }

  protected searchByIdManual(): void {
    this.findArticulo(this.searchIdManual);
  }

  private findArticulo(id: number | null): void {
    this.selectedArticulo = PRODUCTS.find((product) => product.id === id);
    this.searched = true;
  }
}
```

Puntos clave para los alumnos:

- `selectedArticulo` guarda el producto encontrado. Su tipo es `Product | undefined` porque `find()` devuelve `undefined` cuando ningún elemento cumple la condición.
- `searched` indica si ya se ha buscado alguna vez, para distinguir «todavía no se ha buscado» de «se buscó y no existe».

- `findArticulo()` es `private` y contiene la lógica común: busca en `PRODUCTS` con `find()` y actualiza `selectedArticulo` y `searched`. Los métodos que usa la plantilla (`search()`, `searchById()` y `searchByIdManual()`) acaban llamándolo (ver el paso 14).
- Aunque el componente usa `OnPush`, no hacen falta signals: los eventos de la plantilla (`click`, `keyup.enter`, `input`) marcan el componente para revisarlo, y Angular vuelve a pintar la vista con los nuevos valores de las propiedades.

8. Forma 1: variable de plantilla (`#idInput`)

Se marca el `<input>` con la variable de plantilla `#idInput`. Desde cualquier punto de la plantilla, `idInput` da acceso al elemento HTML, y su valor se lee con `idInput.value` y se pasa como parámetro al método:

· **`features/products/products.html`**

```
<label for="product-id">Id del artículo</label>
<input #idInput id="product-id" type="number" min="1" (keyup.enter)="search(idInput.value)" />
<button type="button" (click)="search(idInput.value)">Buscar</button>
```

- `idInput.value` es siempre un `string` (aunque el input sea `type="number"`), por eso `search(id: string)` lo convierte con `Number(id)` antes de buscar.
- `(keyup.enter)` es un evento con filtro de tecla: solo se dispara al pulsar Enter. `(click)` hace lo mismo desde el botón.
- El componente no guarda lo que se escribe: solo recibe el valor cuando se pulsa Buscar o Enter.

9. Forma 2: two-way data binding con `[(ngModel)]`

`[(ngModel)]="searchId"` enlaza el input con la propiedad `searchId` en los dos sentidos: lo que se escribe se guarda en la propiedad y, si la propiedad cambia, el input se actualiza.

· **`features/products/products.html`**

```
<label for="product-id-binding">Id del artículo</label>
<input id="product-id-binding" type="number" min="1" [(ngModel)]="searchId"
(keyup.enter)="searchById()" />
<button type="button" (click)="searchById()">Buscar</button>
```

- Hay que añadir `FormsModule` al array `imports` del componente; si no, Angular no reconoce `ngModel`.
- Con un input `type="number"`, `ngModel` ya entrega un `number` (o `null` si el input está vacío). Por eso `searchId` es de tipo `number | null` y no hace falta convertir nada.
- `searchById()` no recibe parámetros: lee directamente `this.searchId`.
- A la sintaxis `[( )]` se la conoce como «banana in a box»: combina `[ ]` (propiedad → vista) y `( )` (vista → propiedad).

10. Forma 3: two-way data binding manual (`[value]` + `(input)`)

Se consigue lo mismo que con `ngModel`, pero escribiendo a mano los dos sentidos del enlace y sin necesitar `FormsModule`:

- **`features/products/products.html`**

```
<label for="product-id-manual">Id del artículo</label>
<input
  id="product-id-manual"
  type="number"
  min="1"
  [value]="searchIdManual"
  (input)="updateSearchIdManual($event)"
  (keyup.enter)="searchByIdManual()"
/>
<button type="button" (click)="searchByIdManual()">Buscar</button>
```

- **`features/products/products.ts`**

```
protected updateSearchIdManual(event: Event): void {
  const value = (event.target as HTMLInputElement).value;
  this.searchIdManual = value === '' ? null : Number(value);
}
```

- `[value]="searchIdManual"` es un property binding: lleva el valor de la propiedad al input (propiedad → vista).
- `(input)` se dispara cada vez que cambia el contenido del input y llama a `updateSearchIdManual($event)`, que guarda el valor en la propiedad (vista → propiedad).
- `$event` es el objeto `Event` del navegador. `event.target` es de tipo `EventTarget | null`, así que se indica con `as HTMLInputElement` que es un input para poder leer `.value` sin usar `any`.
- Como `.value` es un `string`, se convierte a número. Si el input está vacío se guarda `null`, igual que hace `ngModel`.
- Es lo que `[(ngModel)]` hace por dentro, así que esta forma sirve para entender cómo funciona el two-way data binding.

11. Mostrar el resultado con `@if` / `@else if`

Según el resultado de la búsqueda, la plantilla muestra la tarjeta del producto, un aviso o nada:

· **`features/products/products.html`**

```
<div aria-live="polite">
  @if (selectedArticulo) {
    <article class="product-card">
      <img
        [ngSrc]="selectedArticulo.image"
        [alt]="selectedArticulo.title"
        width="220"
        height="220"
        class="product-card__image"
      />
      <h2 class="product-card__title">{{ selectedArticulo.title }}</h2>
      <p class="product-card__category">{{ selectedArticulo.category }}</p>
      <p class="product-card__price">{{ selectedArticulo.price }} €</p>
      <p class="product-card__rating">
        ★ {{ selectedArticulo.rating.rate }} ({{ selectedArticulo.rating.count }} valoraciones)
      </p>
    </article>
  } @else if (searched) {
    <p class="product-search__empty">No existe ningún artículo con ese id.</p>
  }
</div>
```

- `@if (selectedArticulo)`: si se encontró un producto, se muestra su tarjeta (la misma de la primera parte, ahora con `selectedArticulo` en vez de `product`). Dentro del bloque, TypeScript ya sabe que `selectedArticulo` no es `undefined`.
- `@else if (searched)`: si se buscó y no se encontró, se muestra el aviso.
- Si todavía no se ha buscado, no se muestra nada.
- `aria-live="polite"` hace que los lectores de pantalla anuncien el resultado cuando cambia.

12. Explicación de cada forma en la página

Para que al ejecutar el proyecto se vea cómo funciona cada forma, cada búsqueda se colocó en su propio recuadro (`<section>`) dentro del `div` con `role="search"`. Cada recuadro tiene:

- Un `<h2>` con el nombre de la forma. La `section` usa `aria-labelledby` con el id de su `h2`.
- Los controles: `label`, `input` y botón Buscar.
- Un párrafo que explica cómo funciona.
- Un bloque `<pre><code>` con el fragmento de HTML. Dentro del bloque, `<` y `>` se escriben como `<` y `>` para que el navegador los muestre como texto y no los interprete como etiquetas.

Recuadro de la forma 1 (los otros dos siguen el mismo esquema):

- **features/products/products.html**

```
<!-- 1. Variable de plantilla -->
<section class="search-method" aria-labelledby="method-template-var">
  <h2 id="method-template-var" class="search-method__title">1. Variable de plantilla</h2>
  <div class="search-method__controls">
    <label for="product-id">Id del artículo</label>
    <input #idInput id="product-id" type="number" min="1"
           (keyup.enter)="search(idInput.value)" />
    <button type="button" (click)="search(idInput.value)">Buscar</button>
  </div>
  <p class="search-method__text">
    Se marca el input con <code>#idInput</code> y se lee su valor con
    <code>idInput.value</code>, que se pasa como parámetro a <code>search()</code>.
    El método lo convierte a número con <code>Number()</code>.
  </p>
  <pre class="search-method__code"><code>&lt;input #idInput (keyup.enter)="search(idInput.value)"
/&gt;
&lt;button (click)="search(idInput.value)"&gt;Buscar&lt;/button&gt;</code></pre>
</section>
```

En `products.css` se añadieron los estilos de los recuadros. Gracias a `flex-wrap` y a `flex: 1 1 280px`, los tres recuadros se ven uno al lado del otro y, en pantallas estrechas, se colocan uno debajo de otro:

- **features/products/products.css**

```
.product-search {
  display: flex;
  flex-wrap: wrap;
  align-items: stretch;
  gap: 1rem;
  margin-bottom: 1rem;
}

.search-method {
  flex: 1 1 280px;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  padding: 1rem;
  border: 1px solid #d0d0d0;
  border-radius: 8px;
}
```

Por último, se añadió en el HTML un comentario (`<!-- ... -->`) encima de cada `div` explicando cómo se hace esa parte.

13. Etiquetas y atributos de accesibilidad

Algunas etiquetas y atributos del HTML no cambian lo que se ve en pantalla: sirven para la **accesibilidad**, es decir, para que las personas que usan lectores de pantalla u otras tecnologías de apoyo entiendan cómo está organizada la página y se enteren de los cambios. En el componente se usan dos: `section` con `aria-labelledby` y `div` con `aria-live`.

La etiqueta `section` con `aria-labelledby`

`<section>` agrupa un bloque de contenido relacionado. El atributo `aria-labelledby` le da un nombre accesible: se pone el `id` de otro elemento de la página (normalmente su título) y el lector de pantalla usa el texto de ese elemento como nombre de la sección:

```
<section aria-labelledby="products-title">
  <h1 id="products-title" class="products-title">Buscar producto</h1>
  ...
</section>

<section class="search-method" aria-labelledby="method-ngmodel">
  <h2 id="method-ngmodel" class="search-method__title">2. Two-way binding con ngModel</h2>
  ...
</section>
```

- El valor de `aria-labelledby` tiene que coincidir con el `id` del título. Por eso cada `h1` y `h2` tiene un `id` único.
- Así, al recorrer la página, el lector anuncia, por ejemplo, «región 2. Two-way binding con ngModel» en vez de una sección sin nombre.
- Se usa `aria-labelledby` en vez de `aria-label` porque el título ya se ve en la página: se reutiliza su texto en lugar de escribirlo dos veces.
- De forma parecida, `role="search"` en el `div` de las búsquedas indica que esa zona es un buscador.

La etiqueta `div` con `aria-live`

`aria-live` convierte el `div` en una «región viva»: cuando su contenido cambia, el lector de pantalla lee el contenido nuevo aunque el usuario no haya movido el foco hasta allí:

```
<div aria-live="polite">
  @if (selectedArticulo) {
    <article class="product-card">...</article>
  } @else if (searched) {
    <p class="product-search__empty">No existe ningún artículo con ese id.</p>
  }
</div>
```

- Sin `aria-live`, una persona ciega que pulsa Buscar no sabría si ha aparecido un producto o el aviso de que no existe, porque el foco sigue en el botón.
- `polite` espera a que el lector termine lo que está diciendo antes de anunciar el cambio. El otro valor habitual, `assertive`, interrumpe al momento y se reserva para avisos urgentes.
- El `div` con `aria-live` debe existir siempre en la página: lo que cambia es su contenido (por eso está fuera del `@if`). Si el propio `div` apareciera y desapareciera, algunos lectores no anunciarían nada.

14. Formas de definir las variables y los métodos de un componente

Un componente es una clase de TypeScript, así que sus variables (propiedades) y métodos se pueden declarar con un **modificador de acceso** delante, que indica desde dónde se pueden usar: `public`, `protected`, `private` o sin poner nada.

- **Ejemplo**

```
export class Ejemplo {
  titulo = 'Productos';                              // sin modificador: es public
  public total = 0;                                  // public explícito
  protected searched = false;                        // la clase y su plantilla
  private cache: Product[] = [];                     // solo dentro de la clase

  mostrar(): void {}                                 // método public
  protected search(id: string): void {}              // se puede llamar desde la plantilla
  private findArticulo(id: number | null): void {}   // solo desde otros métodos
}
```

- **Sin poner nada**: en TypeScript, si no se escribe ningún modificador, el miembro es `public`. `titulo` y `total` funcionan exactamente igual.
- **`public`**: se puede usar desde cualquier sitio: desde la propia clase, desde su plantilla y desde fuera (otras clases, o un componente padre que acceda a este componente).
- **`protected`**: se puede usar desde la clase, desde las clases que hereden de ella y, en Angular, **desde la plantilla del componente**. Desde fuera no es accesible. Es la opción recomendada para lo que solo usa la plantilla, porque deja claro que no forma parte de la «API pública» del componente.
- **`private`**: solo se puede usar desde dentro de la propia clase. **No se puede usar en la plantilla**: si se intenta, `ng build` da un error del tipo «Property 'findArticulo' is private and only accessible within class 'Products'». Se usa para la lógica interna.

Modificador	En la clase	En la plantilla	En subclases	Desde fuera
sin modificador	Sí	Sí	Sí	Sí (equivale a `public`)
`public`	Sí	Sí	Sí	Sí
`protected`	Sí	Sí	Sí	No
`private`	Sí	No	No	No

En el componente `Products` se sigue esta regla:

- Las propiedades y los métodos que usa la plantilla son `protected`: `selectedArticulo`, `searched`, `searchId`, `searchIdManual`, `search()`, `searchById()`, `updateSearchIdManual()` y `searchByIdManual()`.
- `findArticulo()` es `private`, porque solo lo llaman los otros métodos de la clase y la plantilla no necesita verlo.
- Ningún miembro es `public`: ningún otro componente necesita acceder a `Products`.

Además del modificador de acceso, hay otras formas de declarar una propiedad:

- **Con el tipo explícito**: `protected searchId: number | null = null;`. Hace falta cuando el tipo no se puede deducir del valor inicial (aquí, con `null` solo, TypeScript no sabría que también puede ser un número).

- **Con inferencia de tipo**: `protected searched = false;`. TypeScript deduce que es `boolean` a partir del valor inicial, así que no hace falta escribirlo.
- **Con `readonly`**: `protected readonly product: Product = PRODUCTS[10];` (primera parte). Solo se puede asignar al declararla o en el constructor; después, cambiar su valor da un error de compilación.

En los métodos se indica el tipo de cada parámetro y el tipo que devuelven, por ejemplo `search(id: string): void` (`void` significa que no devuelve nada).

Los modificadores de acceso solo existen en TypeScript: el compilador los comprueba, pero desaparecen al generar el JavaScript final.

Tercera parte: estado y reactividad con zoneless (03_reactividad)

En la tercera parte (commit `03_reactividad`) se ve qué hace Angular para repintar la vista cuando la aplicación funciona **sin zone.js** (zoneless), que es lo predeterminado en los proyectos nuevos desde Angular 21 (la versión de este proyecto). Se hace en dos fases:

1. **Con un número** (pasos 15 a 17): el componente `Reactividad` tiene un dato en una propiedad normal (`dato1`) y otro en un signal (`dato2`), con los mismos botones para incrementarlos. Así se introducen los signals con el caso más sencillo posible.
2. **Con un array de productos** (pasos 18 a 25): dos componentes con los mismos botones y la misma plantilla, que muestran un listado y permiten dar de alta productos nuevos:
    - `ProductsArray`: guarda los productos en un **array normal** (una propiedad de la clase, como hasta ahora).
    - `ProductsSignal`: guarda los productos en un **signal**.

En los dos casos, la única diferencia entre una versión y otra es cómo se guarda y se modifica el estado.

15. Zoneless: cuándo vuelve a pintar Angular la vista

Este proyecto funciona **sin zone.js** (zoneless): en `package.json` no está `zone.js` y `app.config.ts` no lo configura, porque desde Angular 21 es lo predeterminado.

Antes, zone.js interceptaba todo lo asíncrono del navegador (eventos, `setTimeout`, promesas, peticiones HTTP...) y, al terminar cada cosa, Angular revisaba toda la aplicación por si algo había cambiado. Sin zone.js, Angular **no vigila las propiedades ni los arrays**: solo revisa la vista cuando algo le avisa de que hay cambios. Esos avisos son:

1. **Un evento declarado en la plantilla**: `(click)`, `(input)`, `(keyup.enter)`... Antes de ejecutar el método, Angular marca el componente para revisarlo.
2. **Cambiar el valor de un signal** que se lee en la plantilla.
3. El pipe `async`, cuando el observable emite un valor.
4. Un `input()` que recibe un valor nuevo desde el componente padre.
5. Llamar a mano a `markForCheck()` de `ChangeDetectorRef`.

Por eso funcionaba el buscador de la segunda parte sin signals: `selectedArticulo` y `searched` se cambian siempre dentro de un `(click)` o un `(keyup.enter)` (aviso 1). Los pasos siguientes muestran qué pasa cuando un dato cambia **fuera** de un evento de la plantilla.

16. Componente `Reactividad`: un número sin signal y con signal

Se creó el componente en `features/reactividad` (`reactividad.ts`, `reactividad.html` y `reactividad.css`). Tiene dos datos numéricos con el valor inicial 1: `dato1` es una propiedad normal y `dato2` es un signal.

```
import { ChangeDetectionStrategy, Component, signal } from '@angular/core';

@Component({
  selector: 'app-reactividad',
  templateUrl: './reactividad.html',
  styleUrl: './reactividad.css',
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class Reactividad {
  // Sin signal: propiedad numérica normal.
  protected dato1 = 1;

  // Con signal: signal numérico. Se lee llamándolo como una función: dato2().
  protected readonly dato2 = signal(1);

  // Desde un (click): el evento de la plantilla avisa a Angular y la vista se repinta.
  protected incrementDato1(): void {
    this.dato1++;
    console.log('Sin signal - click: dato1 =', this.dato1);
  }

  // Dentro de un setTimeout: sin zone.js nadie avisa a Angular de este cambio.
  // El valor cambia (se ve en la consola), pero la vista no se repinta hasta el
  // siguiente evento de plantilla, por ejemplo el botón «Revisar componente».
  protected incrementDato1WithTimeout(): void {
    setTimeout(() => {
      this.dato1++;
      console.log('Sin signal - setTimeout: dato1 =', this.dato1);
    }, 2000);
  }

  // update() recibe el valor actual y devuelve el nuevo. Al cambiar el valor del
  // signal, Angular se entera y repinta la vista.
  protected incrementDato2(): void {
    this.dato2.update((value) => value + 1);
    console.log('Con signal - click: dato2 =', this.dato2());
  }

  // Dentro de un setTimeout: el signal avisa a Angular él mismo, así que la vista
  // se repinta sola a los 2 segundos, aunque la aplicación funcione sin zone.js.
  protected incrementDato2WithTimeout(): void {
    setTimeout(() => {
      this.dato2.update((value) => value + 1);
      console.log('Con signal - setTimeout: dato2 =', this.dato2());
    }, 2000);
  }
```

```
  // No cambia nada: solo sirve para provocar un evento de plantilla (click),
  // que obliga a Angular a revisar todo el componente y pintar lo que haya cambiado.
  protected checkView(): void {}
}
```

Puntos clave para los alumnos:

- `signal(1)` crea un signal con el valor inicial 1. Un signal es un «contenedor» de un valor que **avisa a Angular cuando ese valor cambia**. TypeScript deduce el tipo (`number`) a partir del valor inicial; también se puede escribir explícitamente: `signal<number>(1)`.
- **Para leer un signal se llama como una función**: `this.dato2()` en la clase y `{{ dato2() }}` en la plantilla. Al leerlo en la plantilla, Angular apunta que la vista depende de ese signal, y cuando el signal cambie sabrá que tiene que repintarla.
- **Para cambiar un signal** no se asigna con `=`. Se usa `set()`, que recibe el valor nuevo (`this.dato2.set(5)`), o `update()`, que recibe una función que calcula el valor nuevo a partir del actual (`this.dato2.update((value) => value + 1)`).
- `dato2` es `readonly` porque la propiedad siempre apunta al mismo signal: lo que cambia es el valor que hay dentro. `dato1` no puede ser `readonly`, porque `this.dato1++` le asigna un valor nuevo.
- `checkView()` es un método vacío: su único objetivo es que el botón «Revisar componente» tenga un `(click)`, que obliga a Angular a revisar todo el componente (aviso 1 del paso 15).
- Cada método escribe el valor en la consola, para comprobar que el dato cambia aunque la vista no lo muestre.

La plantilla muestra los dos datos en dos `div`, uno con el título «Sin signal» y otro con «Con signal», cada uno con sus dos botones. El botón «Revisar componente» es común a los dos:

```
<section aria-labelledby="reactividad-title">
  <h2 id="reactividad-title" class="reactividad-title">Reactividad</h2>

  <button type="button" class="reactividad-check" (click)="checkView()">Revisar componente</button>

  <div class="reactividad-cases">
    <div class="reactividad-case" role="group" aria-labelledby="sin-signal-title">
      <h3 id="sin-signal-title" class="reactividad-case__title">Sin signal</h3>
      <p class="reactividad-case__value" aria-live="polite">dato1: {{ dato1 }}</p>
      <div class="reactividad-case__actions">
        <button type="button" (click)="incrementDato1()">Incrementar</button>
        <button type="button" (click)="incrementDato1WithTimeout()">Incrementar con setTimeout (2
s)</button>
      </div>
    </div>

    <div class="reactividad-case" role="group" aria-labelledby="con-signal-title">
      <h3 id="con-signal-title" class="reactividad-case__title">Con signal</h3>
      <p class="reactividad-case__value" aria-live="polite">dato2: {{ dato2() }}</p>
      <div class="reactividad-case__actions">
        <button type="button" (click)="incrementDato2()">Incrementar</button>
        <button type="button" (click)="incrementDato2WithTimeout()">Incrementar con setTimeout (2
s)</button>
      </div>
    </div>
  </div>
```

```
</section>
```

- Cada `div` de un caso tiene `role="group"` y `aria-labelledby` con el id de su `h3`, para que los lectores de pantalla lo anuncien como «grupo Sin signal» o «grupo Con signal» (ver el paso 13).
- El párrafo con el valor tiene `aria-live="polite"`, para que el lector de pantalla anuncie el número nuevo cuando cambia.
- En `reactividad.css`, los dos casos se colocan uno al lado del otro con `flex-wrap` y `flex: 1 1 280px`, igual que los recuadros del buscador (paso 12).

17. Probar `Reactividad`: qué se repinta y qué no

Con `ng serve` y la consola del navegador abierta (F12):

Botón	Sin signal (`dato1`)	Con signal (`dato2`)
Incrementar	La vista se actualiza al momento	La vista se actualiza al momento
Incrementar con setTimeout (2 s)	El valor cambia (sale en la consola), pero **la vista no se actualiza**	**La vista se actualiza sola** a los 2 segundos
Revisar componente	Aparece el valor actual de `dato1`	No cambia nada (ya estaba actualizado)

Por qué ocurre:

- **Incrementar**: los dos cambios ocurren dentro de un `(click)` (aviso 1 del paso 15), así que Angular revisa el componente y pinta los dos valores.
- **Incrementar con setTimeout**: el `(click)` ocurre al pulsar, pero el cambio ocurre 2 segundos después, cuando Angular ya ha revisado el componente. Para `dato1`, nadie avisa a Angular de ese cambio. Para `dato2`, el propio signal avisa (aviso 2 del paso 15).
- **Revisar componente**: su `(click)` hace que Angular revise el componente y pinte el valor que `dato1` tiene en ese momento.

Un experimento útil: pulsar tres veces «Incrementar con setTimeout» en «Sin signal», esperar y comprobar que en la consola `dato1` llega a 4 mientras la vista sigue mostrando 1. Al pulsar «Revisar componente», la vista pasa de 1 a 4 de golpe.

La conclusión: con zoneless, **una propiedad normal solo se refleja en la vista si cambia dentro de un evento de la plantilla**. Un signal avisa a Angular él mismo cada vez que cambia, esté donde esté el código que lo modifica. Los pasos siguientes comprueban lo mismo con un array de productos.

18. Cambios previos: `PRODUCTS` con 5 productos

Para que los listados sean cortos y se vea bien cuándo aparece un producto nuevo, en `core/mocks/products.mock.ts` se comentó el array completo de 20 productos y se definió un `PRODUCTS` nuevo solo con los 5 primeros:

```
// Array completo con los 20 productos, comentado: ahora solo se usan los 5 primeros.
/*
export const PRODUCTS: Product[] = [
  ...los 20 productos...
];
*/

// Solo los 5 primeros productos.
export const PRODUCTS: Product[] = [
  { id: 1, /* ... */ },
  // ...
  { id: 5, /* ... */ }
];
```

- Con 5 productos, cada listado ocupa exactamente una fila de 5 tarjetas, y un producto añadido aparece en una fila nueva.
- El buscador de la segunda parte (`Products`) también usa `PRODUCTS`, así que ahora solo encuentra los ids del 1 al 5.
- Los productos 10 y 11, que se darán de alta con los botones, se copian del array original a variables de cada componente (ver los pasos 21 y 22).

19. Imágenes con `NgOptimizedImage` en modo `fill`

Hasta ahora, las imágenes se declaraban con un tamaño fijo cuadrado:

```
<img [ngSrc]="product.image" [alt]="product.title" width="220" height="220" class="product-
card__image" />
```

Al mostrar más productos, la consola del navegador mostró este aviso:

```
NG02952: The NgOptimizedImage directive (activated on an <img> element with the
`ngSrc="/images/11.png"`) has detected that the aspect ratio of the image does
not match the aspect ratio indicated by the width and height attributes.
Intrinsic image size: 606w x 411h (aspect-ratio: 1.47).
Supplied width and height attributes: 220w x 220h (aspect-ratio: 1).
```

`NgOptimizedImage` comprueba que `width` y `height` tengan la misma proporción que la imagen real, y ninguna imagen de los productos es cuadrada (la 11 mide 606×411, la 1 mide 418×597...). En vez de indicar el tamaño de cada imagen, se usa el **modo** `fill`: la imagen rellena un contenedor que tiene un alto fijo.

```
<div class="product-card__image">
  <img [ngSrc]="product.image" [alt]="product.title" fill [priority]="$index < 5" />
</div>
```

```
/* Contenedor de la imagen: NgOptimizedImage en modo fill ocupa todo el contenedor,
   que necesita position y un alto. object-fit: contain ajusta la imagen sin deformarla,
   tenga la proporción que tenga. */
.product-card__image {
  position: relative;
  height: 180px;
}

.product-card__image img {
  object-fit: contain;
}
```

- Con `fill`, el `<img>` no lleva `width` ni `height`: ocupa todo su contenedor.
- El contenedor necesita `position: relative` (o `absolute` / `fixed`) y un alto; si no, `NgOptimizedImage` avisa de que la imagen no se ve.
- `object-fit: contain` encaja la imagen sin deformarla ni recortarla.
- El cambio se aplicó a los tres componentes que muestran productos: el buscador `Products` (con un alto de 220px, porque muestra una sola tarjeta grande), `ProductsArray` y `ProductsSignal` (180px).

20. Componente `ProductsArray`: listado con `@for`

Se creó el componente en `features/products-array` (`products-array.ts`, `products-array.html` y `products-array.css`). Al principio solo mostraba el listado:

```
@Component({
  selector: 'app-products-array',
  imports: [NgOptimizedImage],
  templateUrl: './products-array.html',
  styleUrl: './products-array.css',
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class ProductsArray {
  // Copia del array PRODUCTS: el push se hace sobre el array del componente
  // y no modifica el array original del mock.
  protected readonly products: Product[] = [...PRODUCTS];
}
```

```
<section aria-labelledby="products-array-title">
  <h2 id="products-array-title" class="products-array-title">Productos con array</h2>

  <ul class="products-array-grid">
    @for (product of products; track product.id) {
      <li>
        <article class="product-card">
          <div class="product-card__image">
            <img [ngSrc]="product.image" [alt]="product.title" fill [priority]="$index < 5" />
          </div>
          <h3 class="product-card__title">{{ product.title }}</h3>
          <p class="product-card__category">{{ product.category }}</p>
          <p class="product-card__price">{{ product.price }} €</p>
          <p class="product-card__rating">★ {{ product.rating.rate }} ({{ product.rating.count }}
valoraciones)</p>
        </article>
      </li>
```

```
    } @empty {
      <li class="products-array-empty">No hay productos.</li>
    }
  </ul>
</section>
```

Puntos clave para los alumnos:

- `@for (product of products; track product.id)` recorre el array y pinta una tarjeta por producto. `track` es obligatorio: indica cómo identificar cada elemento, para que Angular no vuelva a crear en el DOM las tarjetas que no han cambiado. Por eso no puede haber dos productos con el mismo `id`.
- `$index` es la posición del elemento en el array. Se usa para cargar con prioridad (`[priority]`) las 5 imágenes de la primera fila.
- `@empty` se muestra cuando el array está vacío.
- `[...PRODUCTS]` crea una copia del array. Si se escribiera `products = PRODUCTS`, el componente compartiría el mismo array que el mock y, al hacer `push`, el producto nuevo aparecería también en el buscador.
- El listado es una lista `<ul>` / `<li>`, para que los lectores de pantalla anuncien cuántos productos hay. El título es un `<h2>` y el de cada tarjeta un `<h3>`, porque la página ya tiene el `<h1>` del buscador.

Las tarjetas se colocan en una rejilla de 5 columnas con CSS Grid. En pantallas más estrechas se reducen las columnas para que las tarjetas se puedan leer:

```
.products-array-grid {
  display: grid;
  grid-template-columns: repeat(5, minmax(0, 1fr));
  gap: 1rem;
  margin: 0;
  padding: 0;
  list-style: none;
}

@media (max-width: 1100px) {
  .products-array-grid { grid-template-columns: repeat(3, minmax(0, 1fr)); }
}

@media (max-width: 700px) {
  .products-array-grid { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}

@media (max-width: 450px) {
  .products-array-grid { grid-template-columns: minmax(0, 1fr); }
}
```

21. Modificar el estado en un array normal: `push`

Se añadió un botón para dar de alta un producto nuevo. Sus datos ya están definidos en una variable del componente (son los del producto 10 del array original):

```
protected readonly newProduct: Product = {
  id: 10,
  title: 'SanDisk SSD PLUS 1TB Internal SSD - SATA III 6 Gb/s',
  price: 109,
  description: '...',
  category: Category.Electronics,
  image: '/images/10.png',
  rating: { rate: 2.9, count: 470 }
};

protected added = false;

protected addProduct(): void {
  if (this.added) {
    return;
  }
  this.products.push(this.newProduct);
  console.log('ProductsArray - click: array products', this.products);
  this.added = true;
}
```

```
<div class="products-array-actions">
  <button type="button" (click)="addProduct()" [disabled]="added">Dar de alta producto</button>
  <p class="products-array-status" aria-live="polite">
    @if (added) {
      Producto «{{ newProduct.title }}» dado de alta.
    }
  </p>
</div>
```

- `push()` añade el producto al final del **mismo** array: no se crea un array nuevo, se modifica el que ya había.
- `products` es `readonly` y aun así se puede hacer `push`: `readonly` impide asignar otro array a la propiedad (`this.products = [...]` daría error), pero no impide cambiar su contenido.
- `added` evita dar de alta dos veces el mismo producto: el botón se desactiva con `[disabled]="added"`. Si se añadiera dos veces, habría dos productos con `id: 10` y el `track product.id` del `@for` daría un aviso de claves duplicadas.
- El párrafo con `aria-live="polite"` anuncia el alta a los lectores de pantalla (ver el paso 13).

Al pulsar el botón, **la tarjeta nueva aparece en la vista**, aunque `products` no es un signal. Es lo mismo que pasaba con `dato1` en el componente `Reactividad` (pasos 15 a 17): `addProduct()` se ejecuta desde un `(click)` de la plantilla (aviso 1 del paso 15), así que Angular revisa el componente después del clic. Además, el `@for` no compara solo la referencia del array, sino que recorre sus elementos, así que detecta que hay un producto más y crea su tarjeta.

22. El caso que no se repinta: `push` dentro de un `setTimeout`

Si el mismo `push` se hace **fuera** de un evento de plantilla, nadie avisa a Angular y la vista no cambia. Para verlo, se añadió a `ProductsArray` un segundo botón que da de alta el producto 11 dos segundos después, dentro de un `setTimeout`:

```
protected readonly delayedProduct: Product = {
  id: 11,
  title: 'Silicon Power 256GB SSD 3D NAND A55 SLC Cache Performance Boost SATA III 2.5',
  /* ... */
  image: '/images/11.png',
  rating: { rate: 4.8, count: 319 }
};

protected delayedRequested = false;

// El click sí avisa a Angular, pero el push ocurre 2 segundos después, dentro del
// setTimeout. Sin zone.js nadie avisa a Angular de ese cambio: el producto está en
// el array (se ve en la consola), pero la vista no se repinta hasta el siguiente
// evento de plantilla.
protected addProductWithTimeout(): void {
  if (this.delayedRequested) {
    return;
  }
  this.delayedRequested = true;
  setTimeout(() => {
    this.products.push(this.delayedProduct);
    console.log('ProductsArray - setTimeout: array products', this.products);
  }, 2000);
}

// No cambia nada: solo sirve para provocar un evento de plantilla (click),
// que hace que Angular revise el componente y pinte lo que haya cambiado.
protected checkView(): void {}
```

```
<div class="products-array-actions">
  <button type="button" (click)="addProductWithTimeout()" [disabled]="delayedRequested">
    Dar de alta con setTimeout (2 s)
  </button>
  <button type="button" (click)="checkView()">Comprobar vista</button>
  <p class="products-array-status" aria-live="polite">
    @if (delayedRequested) {
      Producto «{{ delayedProduct.title }}» se añade al array en 2 s. Mira la consola y pulsa
«Comprobar vista».
    }
  </p>
</div>
```

Qué ocurre al probarlo:

1. Al pulsar «Dar de alta con setTimeout», el botón se desactiva y aparece el mensaje al momento: `delayedRequested` cambia **dentro del click**, y el click sí avisa a Angular.
2. A los 2 segundos se ejecuta el `push`. La consola muestra el array con 6 productos, pero **la tarjeta no aparece**: el `setTimeout` no es un evento de plantilla y nadie avisa a Angular.
3. Al pulsar «Comprobar vista», que no cambia ningún dato, el `(click)` hace que Angular revise el componente y **entonces aparece la tarjeta**.

Lo mismo pasaría con cualquier cambio hecho fuera de un evento de plantilla, por ejemplo dentro de una promesa (`fetch(...).then(...)`) o del `subscribe()` de una petición HTTP. Con zone.js estos casos sí se repintaban, porque zone.js interceptaba el `setTimeout` y las promesas.

23. Componente `ProductsSignal`: el estado en un signal

Se creó un segundo componente en `features/products-signal` (`products-signal.ts`, `products-signal.html` y `products-signal.css`), con la misma plantilla y los mismos botones que `ProductsArray`. La diferencia es que `products` es un **signal** de tipo array de `Product`:

```
import { ChangeDetectionStrategy, Component, signal } from '@angular/core';

@Component({
  selector: 'app-products-signal',
  imports: [NgOptimizedImage],
  templateUrl: './products-signal.html',
  styleUrl: './products-signal.css',
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class ProductsSignal {
  // Signal de tipo array de Product, con los valores iniciales de PRODUCTS (copia del array).
  // En la plantilla se lee llamándolo como una función: products().
  protected readonly products = signal<Product[]>([...PRODUCTS]);
}
```

- Funciona igual que `dato2` en el componente `Reactividad` (paso 16), pero guardando un array en vez de un número.
- `signal<Product[]>(valorInicial)` crea el signal. Aquí el tipo entre `< >` conviene escribirlo, para que quede claro que guarda productos; el valor inicial es una copia de `PRODUCTS`, igual que en `ProductsArray`.
- Para leer el valor, el signal se llama como una función: `products()`.
- En la plantilla, el `@for` recorre `products()` (con paréntesis):

```
@for (product of products(); track product.id) {
  ...
}
```

Al leer `products()` en la plantilla, Angular apunta que esta vista depende de ese signal: cuando el signal cambie, sabrá que tiene que repintarla.

- `products` es `readonly` porque la propiedad siempre apunta al mismo signal: lo que cambia es el valor que hay dentro.

24. Modificar un signal: `update()` con un array nuevo

Con un signal no se hace `push`. Su valor se cambia con `set()` (se le da el valor nuevo) o con `update()` (se calcula el valor nuevo a partir del actual):

```
// Indican si ya se ha pulsado cada botón, para no añadir dos veces el mismo producto
// (el @for usa track product.id y no puede haber dos productos con el mismo id).
// Se cambian dentro del click, así que no necesitan ser signals.
protected added = false;
protected delayedRequested = false;
```

```
// Con un signal no se hace push: update() recibe el array actual y devuelve uno NUEVO
// con el producto añadido. Al cambiar el valor del signal, Angular se entera y repinta.
// (Un push dentro de update() devolvería el mismo array y el signal no avisaría del cambio).
protected addProduct(): void {
  if (this.added) {
    return;
  }
  this.products.update((products) => [...products, this.newProduct]);
  console.log('ProductsSignal - click: array products', this.products());
  this.added = true;
}

// El mismo alta, pero 2 segundos después, dentro de un setTimeout. A diferencia de
// ProductsArray, aquí no hace falta ningún evento más: el cambio del signal avisa a Angular
// y la tarjeta aparece sola, aunque la aplicación funcione sin zone.js (zoneless).
protected addProductWithTimeout(): void {
  if (this.delayedRequested) {
    return;
  }
  this.delayedRequested = true;
  setTimeout(() => {
    this.products.update((products) => [...products, this.delayedProduct]);
    console.log('ProductsSignal - setTimeout: array products', this.products());
  }, 2000);
}
```

Puntos clave para los alumnos:

- `update()` recibe una función que recibe el valor actual (`products`) y devuelve el valor nuevo. `[...products, this.newProduct]` crea un **array nuevo** con los productos anteriores más el nuevo.
- **Hay que devolver un array nuevo.** El signal compara el valor anterior con el nuevo y solo avisa si son distintos. Si se hiciera `this.products.update((products) => { products.push(p); return products; })`, se devolvería el mismo array, el signal no detectaría el cambio y no avisaría a Angular. Es el error más habitual al empezar con signals.
- Lo mismo se podría escribir con `set()`: `this.products.set([...this.products(), this.newProduct])`.
- **Con el `setTimeout`, la tarjeta aparece sola a los 2 segundos.** Cambiar el valor del signal es un aviso para Angular (aviso 2 del paso 15), da igual que ocurra dentro de un click, de un `setTimeout` o de una petición HTTP. Por eso `ProductsSignal` no necesita el botón «Comprobar vista».
- `added` y `delayedRequested` siguen siendo propiedades normales: solo cambian dentro del `(click)`, que ya avisa a Angular. Si alguna cambiara dentro del `setTimeout`, también tendría que ser un signal.

25. Comparación y cómo probarla

Los componentes de la tercera parte se montan en `AppComponent`, junto al buscador de la segunda parte:

- **`app.ts`**

```
import { ProductsArray } from './features/products-array/products-array';
import { ProductsSignal } from './features/products-signal/products-signal';
import { Products } from './features/products/products';
import { Reactividad } from './features/reactividad/reactividad';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Products, ProductsArray, ProductsSignal, Reactividad],
  /* ... */
})
export class App {}
```

- **`app.html`**

```
<app-reactividad />
<app-products />
<app-products-array />
<app-products-signal />
<router-outlet />
```

Se puede comentar cualquiera de las etiquetas (`<!-- ... -->`) para mostrar solo los componentes que interesen en cada momento. Si un componente está en `imports` pero su etiqueta está comentada, `ng build` muestra el aviso `NG8113: ... is not used within the template of App`, que no es un error.

	`ProductsArray` (array normal)	`ProductsSignal` (signal)
Declaración	`products: Product[] = [...PRODUCTS]`	`products = signal<Product[]>([...PRODUCTS])`
Lectura en la plantilla	`products`	`products()`
Cómo se añade un producto	`this.products.push(p)` (mismo array)	`this.products.update(ps => [...ps, p])` (array nuevo)
Alta desde un `(click)`	Se repinta (el click avisa a Angular)	Se repinta
Alta dentro de un `setTimeout`	El dato está en el array, pero **no se repinta** hasta el siguiente evento de plantilla	**Se repinta sola**: el signal avisa a Angular

Para probarlo, con `ng serve` y la consola del navegador abierta (F12):

1. Pulsar «Dar de alta producto» en los dos componentes: en los dos aparece la tarjeta del producto 10.

2. Pulsar «Dar de alta con setTimeout (2 s)» en los dos componentes. A los 2 segundos, la consola muestra en los dos casos el array con el producto 11, pero **solo en `ProductsSignal` aparece la tarjeta**.
3. Pulsar «Comprobar vista» en `ProductsArray`: ahora aparece también su tarjeta.

Cada alta escribe en la consola el array completo de productos, empezando por el nombre del componente (`ProductsArray - click: ...`, `ProductsSignal - setTimeout: ...`). Un detalle al mirar la consola: el navegador guarda una **referencia** al array, no una copia. En `ProductsArray`, como siempre es el mismo array, si se despliega un mensaje antiguo después de añadir más productos se ve el contenido actual (la cabecera del mensaje plegado, por ejemplo `(6)`, sí indica cuántos había en ese momento). En `ProductsSignal` no pasa, porque cada `update()` crea un array nuevo.

La conclusión es la misma que con los números del paso 17: con zoneless, una propiedad normal (sea un número o un array) solo se refleja en la vista si cambia dentro de un evento de la plantilla. Un signal avisa a Angular él mismo cada vez que cambia, esté donde esté el código que lo modifica.

Resumen de conceptos TypeScript/Angular tocados

Paso	Concepto
Interfaces `Product` / `Rating`	Definición de tipos, composición de interfaces
`PRODUCTS: Product[]`	Tipado de arrays y objetos literales
`PRODUCTS[10]`	Acceso a elementos de un array por índice
`protected readonly product: Product`	Anotación de tipo explícita en una propiedad de clase
Componente standalone	`imports`, `templateUrl`, `styleUrl` sin `NgModule`
`ChangeDetectionStrategy.OnPush`	Estrategia de detección de cambios
`NgOptimizedImage`	Directiva para optimizar imágenes estáticas
`enum Category`	Enumerado de tipo string para restringir los valores posibles de un campo
`#idInput`	Variable de plantilla para acceder a un elemento HTML desde la plantilla
`(click)` / `(keyup.enter)`	Event binding y filtro de tecla
`[(ngModel)]` + `FormsModule`	Two-way data binding
`[value]` + `(input)`	Two-way data binding manual: property binding + event binding
`$event` / `as HTMLInputElement`	Objeto del evento y aserción de tipo sin usar `any`
`PRODUCTS.find()`	Búsqueda en un array: devuelve el elemento o `undefined`

Paso	Concepto
`Product | undefined`	Tipos unión
`@if` / `@else if`	Control de flujo en la plantilla
`aria-live` / `aria-labelledby`	Accesibilidad: anunciar cambios y asociar cada recuadro con su título
`public` / `protected` / `private`	Modificadores de acceso de propiedades y métodos; sin modificador equivale a `public`
`readonly`	Propiedad que no se puede reasignar después de inicializarla
`NgOptimizedImage` con `fill`	Imagen que rellena un contenedor con alto fijo, sin indicar `width` ni `height`
`@for` / `track` / `$index` / `@empty`	Recorrer un array en la plantilla, identificar cada elemento, posición y caso vacío
`[...PRODUCTS]`	Copiar un array con el operador spread
`push()`	Modificar el contenido de un array sin crear uno nuevo
`[disabled]`	Property binding sobre un atributo del botón
Zoneless	Angular solo revisa la vista cuando algo le avisa: eventos de plantilla, signals, `async`, inputs, `markForCheck()`
`setTimeout` con una propiedad normal	Un cambio fuera de un evento de plantilla no se repinta en zoneless (`dato1`, array `products`)
`signal(1)` / `dato2()`	Crear un signal numérico (tipo inferido) y leer su valor
Método vacío en un `(click)`	Provocar a mano que Angular revise el componente («Revisar componente», «Comprobar vista»)
`signal<Product[]>()` / `products()`	Crear un signal tipado y leer su valor
`update()` / `set()`	Cambiar el valor de un signal; con arrays, siempre devolviendo un array nuevo
`console.log()`	Ver el estado en la consola del navegador

En la tercera parte ya se ha introducido `@for` y los signals. Con esto se puede explicar por qué en una aplicación zoneless conviene guardar en signals el estado que puede cambiar fuera de los eventos de la plantilla.